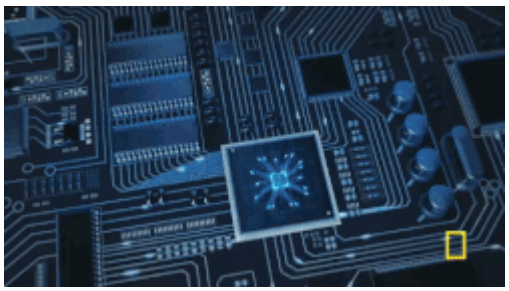
Timer und Interrupts

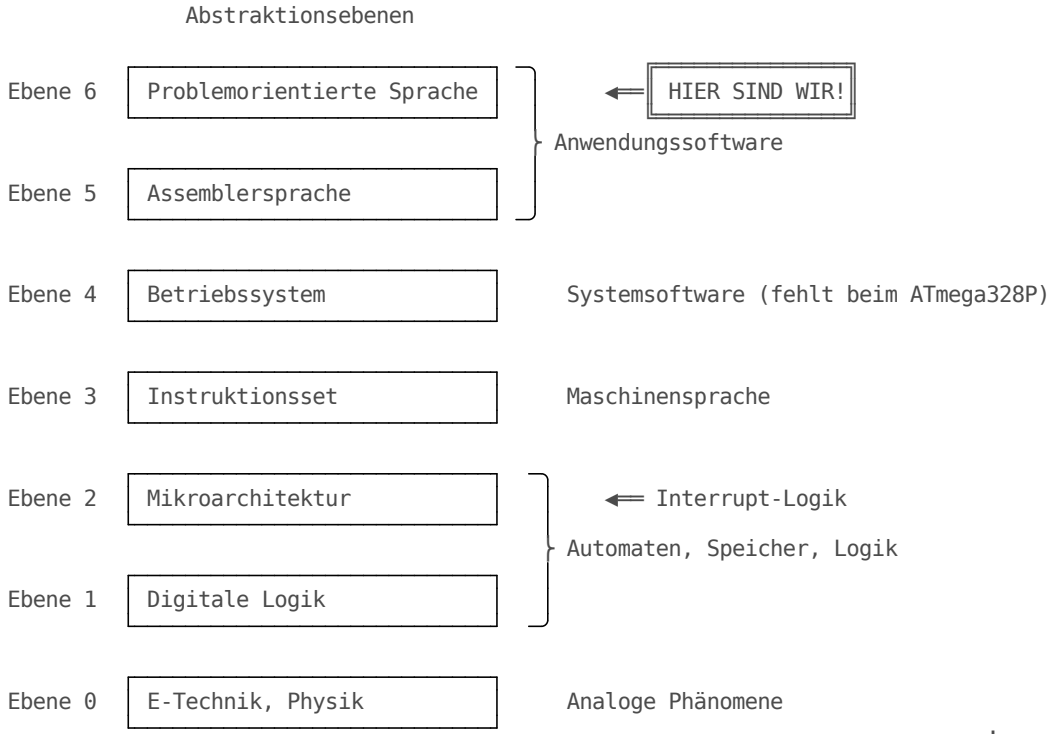
Parameter	Kursinformationen
Veranstaltung:	`Digitale Systeme / Eingebettete Systeme`
Semester:	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Konzepte und Nutzung von Timern und Interrupts`
Link auf GitHub:	https://github.com/TUBAF-IfI-LiaScript/VL_EingebetteteSysteme/blob/master/14_TimerUndInterrupts.md
Autoren:	Sebastian Zug & André Dietrich & Fabian Bär & Claude Code (Opus 5.5)



Lernziele: Nach dieser Vorlesung können Sie

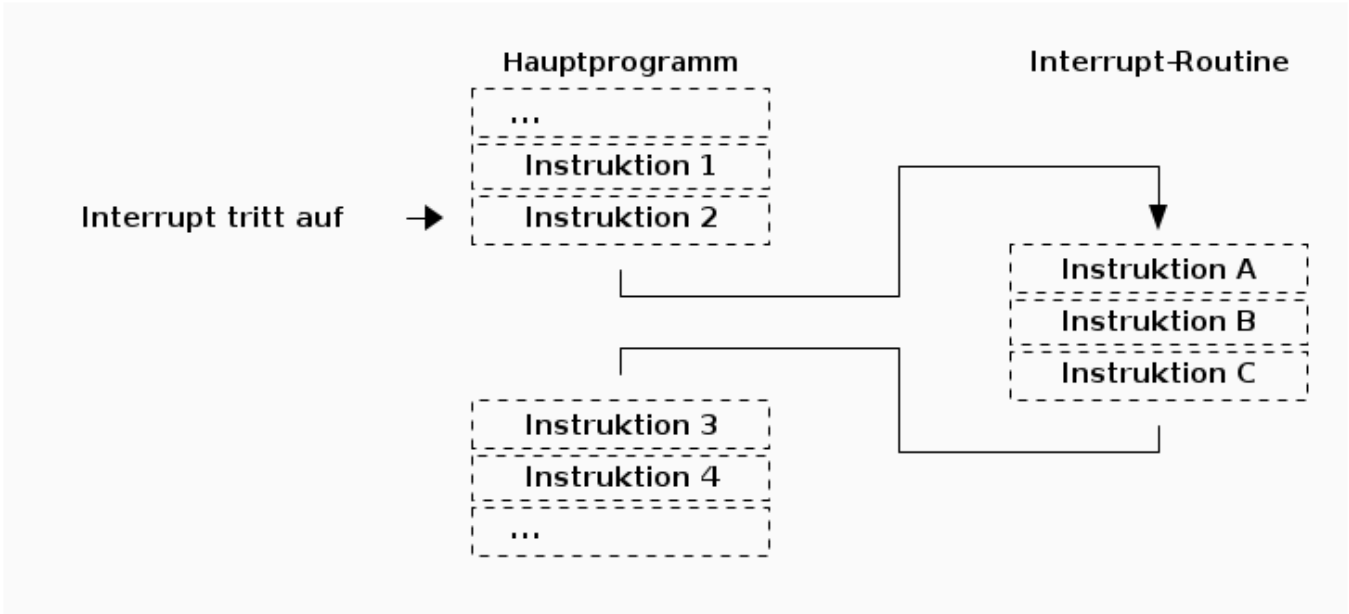
- Polling und Interrupts vergleichen und den Ablauf einer Unterbrechung von der Auslösung bis zum Rücksprung beschreiben,
- beim ATmega328P einen Interrupt über globale und lokale Freigabe aktivieren, eine ISR schreiben und Interruptquellen nennen,
- erklären, warum Variablen, die Hauptprogramm und ISR teilen, als volatile gekennzeichnet und atomar verändert werden müssen,
- den Aufbau eines Timers mit Capture- und Compare-Einheit erklären und die Betriebsarten Normal, CTC und PWM unterscheiden,
- mit Timer 1 ein periodisches Signal oder eine PWM erzeugen und die Frequenz aus Takt, Prescaler und Vergleichswert berechnen.

🗺️ Diese Vorlesung in der Kurslandkarte →



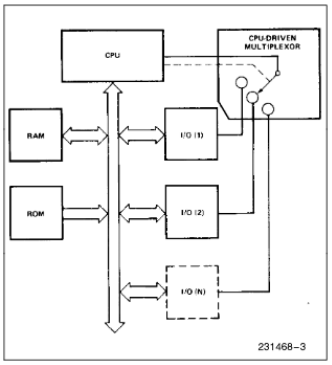
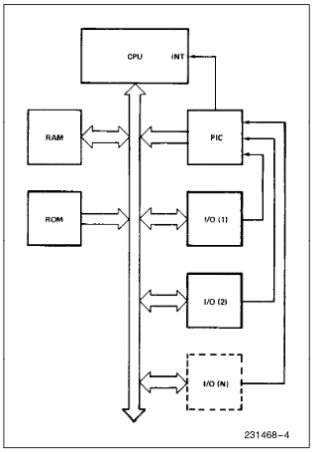
Interrupts

Ein Interrupt beschreibt die kurzfristige Unterbrechung der normalen Programmausführung, um einen, in der Regel kurzen, aber zeitlich kritischen, Vorgang abzuarbeiten.



Beispiele

Trigger	Beispiel
Pin Zustandswechsel	Drücken des Notausbuttons
Kommunikationsschnittstelle	Eintreffen eines Bytes
Analog Comperator	Resultat einer Analog-Comperator Auswertung
Analog-Digital-Wandler	Abschluss einer Wandlung
Timer	Übereinstimmung von Vergleichswert und Zählerwert

	Polling (zyklisches Abfragen)	Interrupts
	 Figure 3a. Polled Method [1]	 Figure 3b. Interrupt Method [1]
Vorteile	• Kein zusätzlicher Hardwareaufwand • Deterministisches Zeitverhalten	• Effiziente Abarbeitung bezogen auf die Auftretenshäufigkeit
Nachteile	• Auslastung des Prozessors • Verzögerung der Reaktion durch periodisches Abarbeitungskonzept	• Zusätzlicher Hardwareaufwand

[1] Firma Intel, Manual Intel 8259, Seite 3, Link

Formen der Unterbrechungsbehandlung

- Traps … *a programmer initiated and expected transfer of control to a special handler routine (software interrupt).*
  - auf x86 Architekturen - INT
  - für AVR – kein eigner OP-Code
- Exceptions … *is an automatically generated trap (coerced rather than that occurs in response to some exceptional condition. requested)*
  - auf x86 – für Division durch 0, fehlerhaften Speicherzugriff, illegal OP Code
  - für AVR – alle Resetquellen
- Interrupts (hardware interrupts) … *are program control interruption based on an external hardware event (external to the CPU)*

Interupt Service Routine = {Trap handling, Exception handling, …}

Ablauf

Schritt	**Beschreibung**
	Normale Programmabarbeitung …
Vorbereitung	• Beenden der aktuelle Instruktion • (Deaktivieren der Interrupts) • Sichern des Registersatzes auf dem Stack
Ausführung	• Realisierung der Interrupt-Einsprung-Routine • Sprung über die Interrupt-Einsprungtabelle zur Interrupt-Behandlungs-Routine • Ausführung der Interrupt-Behandlungs-Routine
Rücksprung	• Wiederherstellen des Prozessorzustandes und des Speichers vom Stack • Sprung in den Programmspeicher mit dem zurückgelesenen PC
	Fortsetzung des Hauptprogrammes …

Notwendige Funktionalität und Herausforderungen:

- Erkenne die Interruptquelle
- Bewerte die Relevanz für das aktuelle Programm
- Unterbreche den Programmablauf transparent und führe die ISR aus
- Beachte unterschiedliche Prioritäten für den Fall gleichzeitig eintreffender Interrupts

> **Merke** ISRs sollten das Hauptprogramm nur kurz unterbrechen! Ein blockierendes Warten ist nicht empfehlenswert! Zudem darf die Abarbeitungsdauer nicht länger sein als die höchste Wiederauftretensfrequenz des Ereignisses.

Umsetzung auf dem AVR

Interrupts müssen individuell aktiviert werden. Dazu dient auf praktisch allen Mikrocontrollern ein zweistufiges System.

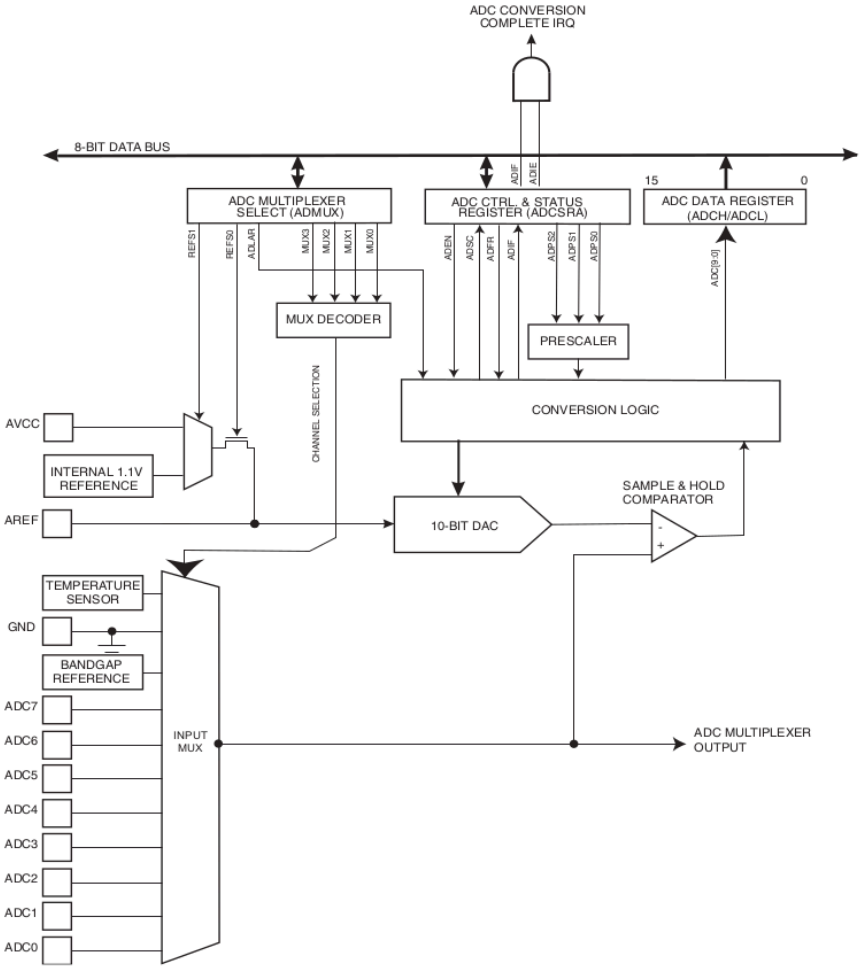
- Die Globale Interruptsteuerung erfolgt über ein generelles CPU-Statusbit, für den AVR Core is dies das I-Bit (Interrupt) im Statusregister (SREG).
- Die jeweilige lokale Interruptsteuerung erlaubt die individuelle Aktivierung über ein Maskenbit jeder Interruptquelle.

Damit können wahlweise Interrupts generell deaktiviert werden während die einzelne Konfiguration unverändert bleibt. Umgekehrt lassen sich spezifische Funktionalitäten ansprechen.

Eine ISR wird demnach nur dann ausgeführt, wenn

- die Interrupts global freigeschaltet sind
- das individuelle Maskenbit gesetzt ist
- der Interrupt (in dem konfigurierten Muster) eintritt

Figure 24-1. Analog to Digital Converter Block Schematic Operation



Analog Digital Converter, Seite 243 [megaAVR]

24.9.2 ADCSRA – ADC Control and Status Register A

Bit	7	6	5	4	3	2	1	0	
(0x7A)	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ADC Konfigurationsregister, Seite 258 [megaAVR]

12.4 Interrupt Vectors in ATmega328 and ATmega328P

Table 12-6. Reset and Interrupt Vectors in ATmega328 and ATmega328P

VectorNo.	Program Address(2)	Source	Interrupt Definition
1	0x0000(1)	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time-out Interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1 COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1 COMPB	Timer/Coutner1 Compare Match B
14	0x001A	TIMER1 OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 Compare Match B
17	0x0020	TIMER0 OVF	Timer/Counter0 Overflow
18	0x0022	SPI, STC	SPI Serial Transfer Complete
19	0x0024	USART, RX	USART Rx Complete
20	0x0026	USART, UDRE	USART, Data Register Empty
21	0x0028	USART, TX	USART, Tx Complete
22	0x002A	ADC	ADC Conversion Complete
23	0x002C	EE READY	EEPROM Ready
24	0x002E	ANALOG COMP	Analog Comparator
25	0x0030	TWI	2-wire Serial Interface
26	0x0032	SPM READY	Store Program Memory Ready

Interrupt-Vector-Tabelle, Seite 74 [megaAVR]

[1]: Firma Microchip, megaAVR® Data Sheet, Seite 74, Link

Im Anschluss wird die Integration der Interrupt-Vektortabelle im Speicher dargestellt.

Der AVR deaktiviert die Ausführung von Interrupts, wenn er eine ISR aktiv ist. Dies kann aber manuell überschrieben werden.

24.9.2 ADCSRA – ADC Control and Status Register A [3]

Bit	7	6	5	4	3	2	1	0	
(0x7A)	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

[3]: Firma Microchip, megaAVR® Data Sheet, Seite 258, Link

ADIF is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, ADIF is cleared by writing a logical one to the flag. Beware that if doing a Read-Modify-Write on ADCSRA, a pending interrupt can be disabled.

> **Merke:** Der „Speichermechanismus“ ein und des selben Interrupts ist ein Bit groß. Von anderen Interrupts, die zwischenzeitlich eintreffen kann jeweils ein Auftreten gespeichert werden.

[megaAVR]: Firma Microchip, megaAVR® Data Sheet, Link

Praktische Interruptprogrammierung

Atomarer Datenzugriff

> **Merke:** Das Hauptprogramm kann grundsätzlich an jeder beliebigen Stelle unterbrochen werden, sofern die Interrupts aktiv sind.

Das bedeutet, dass entsprechende Variablen und Register, die sowohl im Hauptprogramm als auch in Interrupts verwendet werden, mit Sorgfalt zu behandeln sind.

Dies ist umso bedeutsamer, als das ein C Aufruf `port |= 0x03;` in drei Assemblerdirektiven übersetzt wird. An welcher Stelle wäre ein Interrupt kritisch?

```
  IN  r16, port
  ORI r16, 0x03
  OUT port, r16
```

```
// Manuelle Methode
cli(); // Interrupts abschalten
port |= 0x03;
sei(); // Interrupts wieder einschalten

// Per Macros aus dem avr gcc, schöner lesbar und bequemer
// siehe Doku der avr-libc, Abschnitt <util/atomic.h>
ATOMIC_BLOCK(ATOMIC_FORCEON) {
  port |= 0x03;
}
```

Mit Blick auf die Wiederverwendbarkeit des Codes sollte geprüft werden, ob die globalen Interrupts überhaupt aktiviert waren! Die avr-libc hält dafür die Methode `ATOMIC_BLOCK(ATOMIC_RESTORESTATE)` bereit.

Volatile Variablen

`volatile` Variabeln untersagen dem Compiler Annahmen zu deren Datenfluss zu treffen. Mit

volatile.c

```
volatile uint8_t i;

ISR( INT0_vect ){
  i++;
}

int main(){
  ...
  i = 0;
  while( 1 ) {
      Serial.println(i);
  }
}
```

Einführungsbeispiele

Externe Interrupts

Wenden wir das Konzept mal auf einen konkreten Einsatzfall an und lesen die externen Interrupts in einer Schaltung. Dabei sollen Aktivitäten an einem externen Interruptsensiblen Pin überwacht werden.

Aufgabe: Ermitteln Sie mit die PORT zugehörigkeit und die ID der Externen Interrupt PINs `INT0` und `INT1`. Welche Arduino Pin ID gehört dazu?

main.cpp

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/interrupt.h>

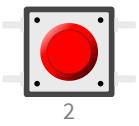
ISR(INT0_vect) {
  PORTB |= (1 << PB5);
}

int main (void) {
    DDRB |= (1 << PB5);
    DDRD &= ~(1 << DDD2);        // Pin als Eingang
    PORTD |= (1 << PD2);      // Pullup-Konfiguration
    EIMSK |= ( 1 << INT0);
    EICRA |= ( 1 << ISC01);
    sei();
    while (1);
    return 0;
}
```

Im Unterschied dazu sei im nachfolgenden Beispiel eine Implementierung des Polling-Mechanismus vorgestellt. Damit soll der Unterschied zwischen dem Interrupt-Modus und der kontinuierlichen Abfrage verdeutlicht werden.

Simulation time: 00:31.153





PORTB: 00000000 DDRB: 00100000 PINB: 00000000

ToggleLED.cpp

```
1  // preprocessor definition
2  #define F_CPU 16000000UL
3
4  #include <avr/io.h>
5  #include <util/delay.h>
6
7  int main (void) {
8      DDRB |= (1 << PB5);
9      int state = 0;
10     // aktives überwachen des Pins
11     while(1) {
12         if (state ^ ((PIND >> PD2) & 1)) {
13             state ^= 1;
14             if (state) {
15               PORTB ^= ( 1 << PB5 );
16             }
17         }
18         _delay_ms(1);
19     }
20     return 0;
21 }
```

```
starting simmulation...
compiling...
ready!
```

Frage: Erklären Sie den Ablauf der Aktualisierung der Zustandsvariablen in Zeile 12 und 13.

Analog Digitalwandler

Das folgende Beispiel nutzt den Analog-Digital-Wandler in einem teilautonomen Betrieb. Innerhalb der Interrupt-Routine wird das Ergebnis ausgewertet und jeweils eine neue Wandlung aktiviert.

Simulation time: 00:19.167



1.0 V

AnalogViaInterrupts.cpp

```
#include <avr/io.h>
#include <avr/interrupt.h>

// Interrupt subroutine for ADC conversion complete interrupt
ISR(ADC_vect) {
    //Serial.println(ADCW);
    if(ADCW >= 600)
        PORTB |= (1 << PB5);
    else
        PORTB &=~(1 << PB5);
    ADCSRA |= (1<<ADSC);
}

int main(void){
    Serial.begin(9600);
    DDRB |= (1 << PB5);
    ADMUX = (1<<REFS0);
    ADCSRA |= (1 << ADPS2) | (1 << ADPS1) | (1<<ADPS0) | (1<<ADEN)|(1
      );
    ADCSRA |= (1<<ADSC);                // Dummy read
    while(ADCSRA & (1<<ADSC));
    ADCSRA |= (1<<ADSC);                // Start conversion "chain"
    (void) ADCW;
    sei();                              // Set the I-bit in SREG

    int i = 0;
    while (1) {
        // Ich rechner fleißig!
        _delay_ms(5000);
    }

    return 0;
}
```

```
starting simmulation...
compiling...
ready!
```

Interrupt-Vektortabelle im AVR Programm

Und warum funktioniert das Ganze? Lassen Sie uns ein Blick hinter die Kulissen werfen.

Folgendes Programm, dass den externen Interrupt 0 auswertet wurde disassembliert.

```
#define F_CPU 16000000UL
```

```
#include <avr/io.h>
#include <avr/interrupt.h>

ISR(INT0_vect) {
  PORTB |= (1 << PB5);
}

int main (void) {
   DDRB |= (1 << PB5);
   DDRD &= ~(1 << DDD2);
   PORTD |= (1 << PD2);
   EIMSK |= ( 1 << INT0);
   EICRA |= ( 1 << ISC01);
   sei();
   while (1);
   return 0;
}
```

main.asm

```
; Interrupt Vector Tabelle mit zugehörigen Sprungfunktionen
00000000 <__vectors>:
   0: 0c 94 34 00   jmp 0x68  ; 0x68 <__ctors_end>
   4: 0c 94 40 00   jmp 0x80  ; 0x80 <__vector_1>
   8: 0c 94 3e 00   jmp 0x7c  ; 0x7c <__bad_interrupt>

  60: 0c 94 3e 00   jmp 0x7c  ; 0x7c <__bad_interrupt>
  64: 0c 94 3e 00   jmp 0x7c  ; 0x7c <__bad_interrupt>

; Initialisierungsroutine für den Controller
00000068 <__ctors_end>:
  68: 11 24         eor r1, r1    ;       0 in R1
  6a: 1f be         out 0x3f, r1  ;       SREG = 0
  6c: cf ef         ldi r28, 0xFF ; 255
  6e: d8 e0         ldi r29, 0x08 ; 8     SP -> 0x8FF
  70: de bf         out 0x3e, r29 ; 62    SPH (0x3e)
  72: cd bf         out 0x3d, r28 ; 61    SPL (0x3d)
  74: 0e 94 4b 00   call  0x96  ; 0x96 <main>
  78: 0c 94 56 00   jmp 0xac  ; 0xac <_exit>

; Restart beim Erreichen eines nicht definierten Interrupts
0000007c <__bad_interrupt>:
  7c: 0c 94 00 00   jmp 0 ; 0x0 <__vectors>

; Interrupt Routine für External Interrupt 0
00000080 <__vector_1>:
  80: 1f 92         push  r1
  82: 0f 92         push  r0
  84: 0f b6         in  r0, 0x3f  ; 63
  86: 0f 92         push  r0
  88: 11 24         eor r1, r1
  ; PORTB |= (1 << PB5);
  8a: 2d 9a         sbi 0x05, 5 ; 5
  8c: 0f 90         pop r0
  8e: 0f be         out 0x3f, r0  ; 63
  90: 0f 90         pop r0
  92: 1f 90         pop r1
  94: 18 95         reti

00000096 <main>:
  ;DDRB |= (1 << PB5);
  96: 25 9a         sbi 0x04, 5 ; 4
  ; DDRD &= ~(1 << DDD2);
  98: 52 98         cbi 0x0a, 2 ; 10
  ; PORTD |= (1 << PORTD2);
  9a: 5a 9a         sbi 0x0b, 2 ; 11
  ; EIMSK |= ( 1 << INT0);
  9c: e8 9a         sbi 0x1d, 0 ; 29
```

```
  ; EICRA |= ( 1 << ISC01);
  9e: 80 91 69 00   lds r24, 0x0069 ; 0x800069 <__DATA_REGION_ORIGIN__+0x9>
  a2: 82 60         ori r24, 0x02 ; 2
  a4: 80 93 69 00   sts 0x0069, r24 ; 0x800069 <__DATA_REGION_ORIGIN__+0x9>
  ; sei();
  a8: 78 94         sei
  aa: ff cf         rjmp  .-2       ; 0xaa <main+0x14>

000000ac <_exit>:
  ac: f8 94         cli

000000ae <__stop_program>:
  ae: ff cf         rjmp  .-2       ; 0xae <__stop_program>
```

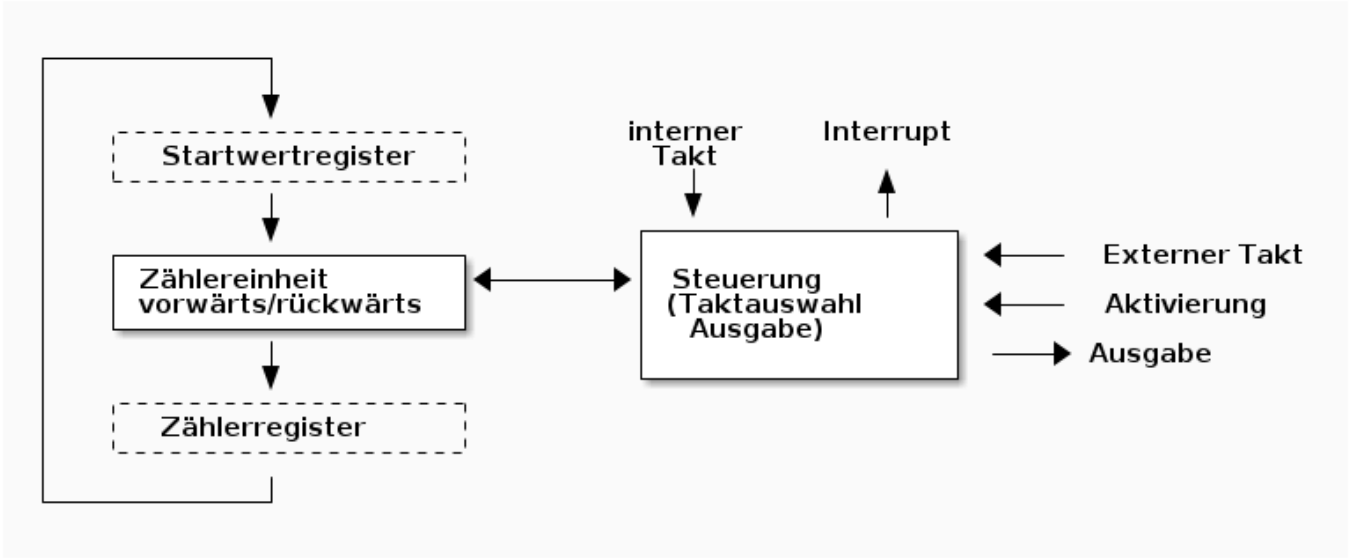
Timer

Aufgaben von Timer/Counter Lösungen in einem Mikrocontroller:

- Zählen von Ereignissen
- Messen von Zeiten, Frequenzen, Phasen, Perioden
- Erzeugen von Intervallen, Pulsfolgen, Interrupts
- Überwachen von Ereignissen und Definition von Zeitstempeln

Basisfunktionalität

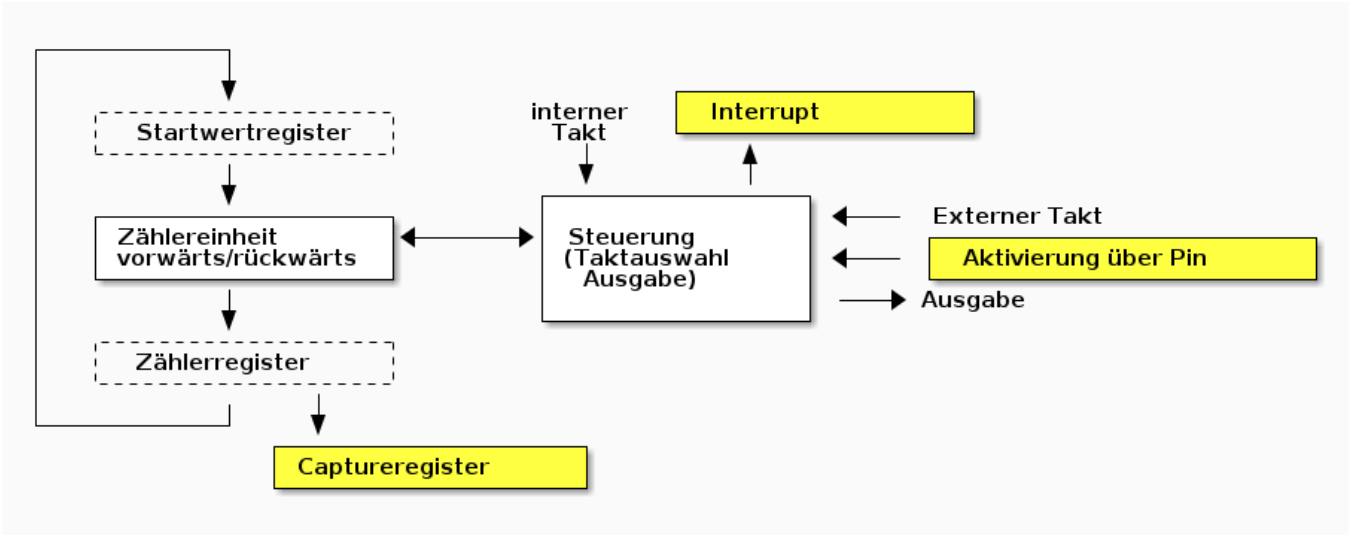
Die Grundstruktur eines Zählerbaustein ergibt sich aus folgenden Komponenten:



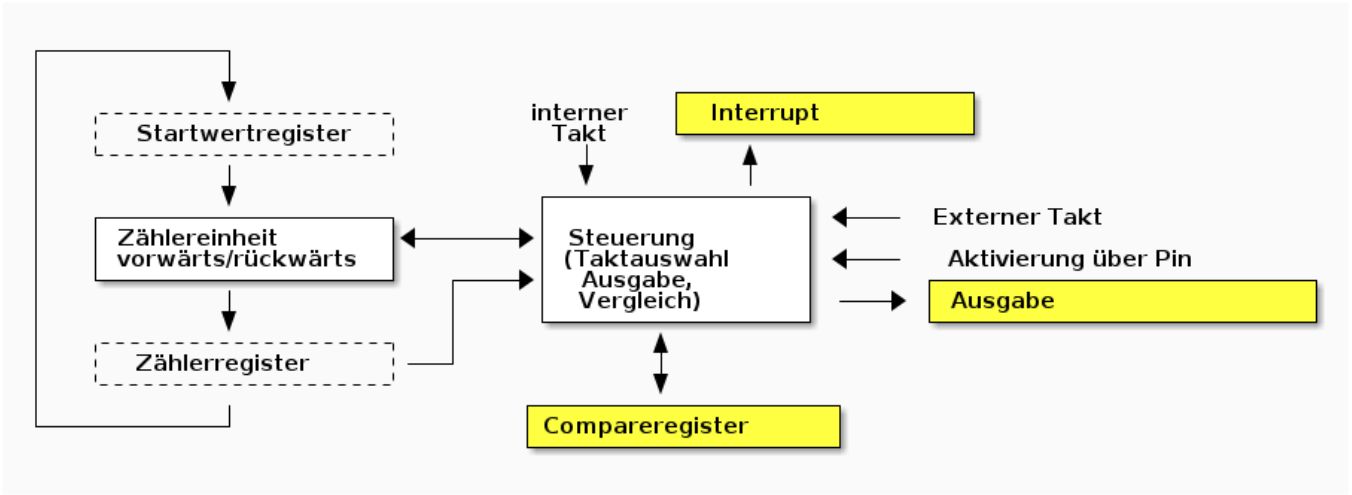
Capture/Compare-Einheiten Nutzen die Basis-Timerimplementierung. Sie können externe Signale aufnehmen und vergleichen, aber beispielsweise auch Pulsmuster erzeugen.

Sie besitzt meist mehrere Betriebsmodi:

- Timer/Zähler-Modus: Aufwärtszählen mit verschiedenen Quellen als Taktgeber. Bei Zählerüberlauf kann ein Interrupt ausgelöst werden.
- Capture-Modus: Beim Auftreten eines externen Signals wird der Inhalt des zugeordneten (laufenden) Timers gespeichert. Auch hier kann ein Interrupt ausgelöst werden.

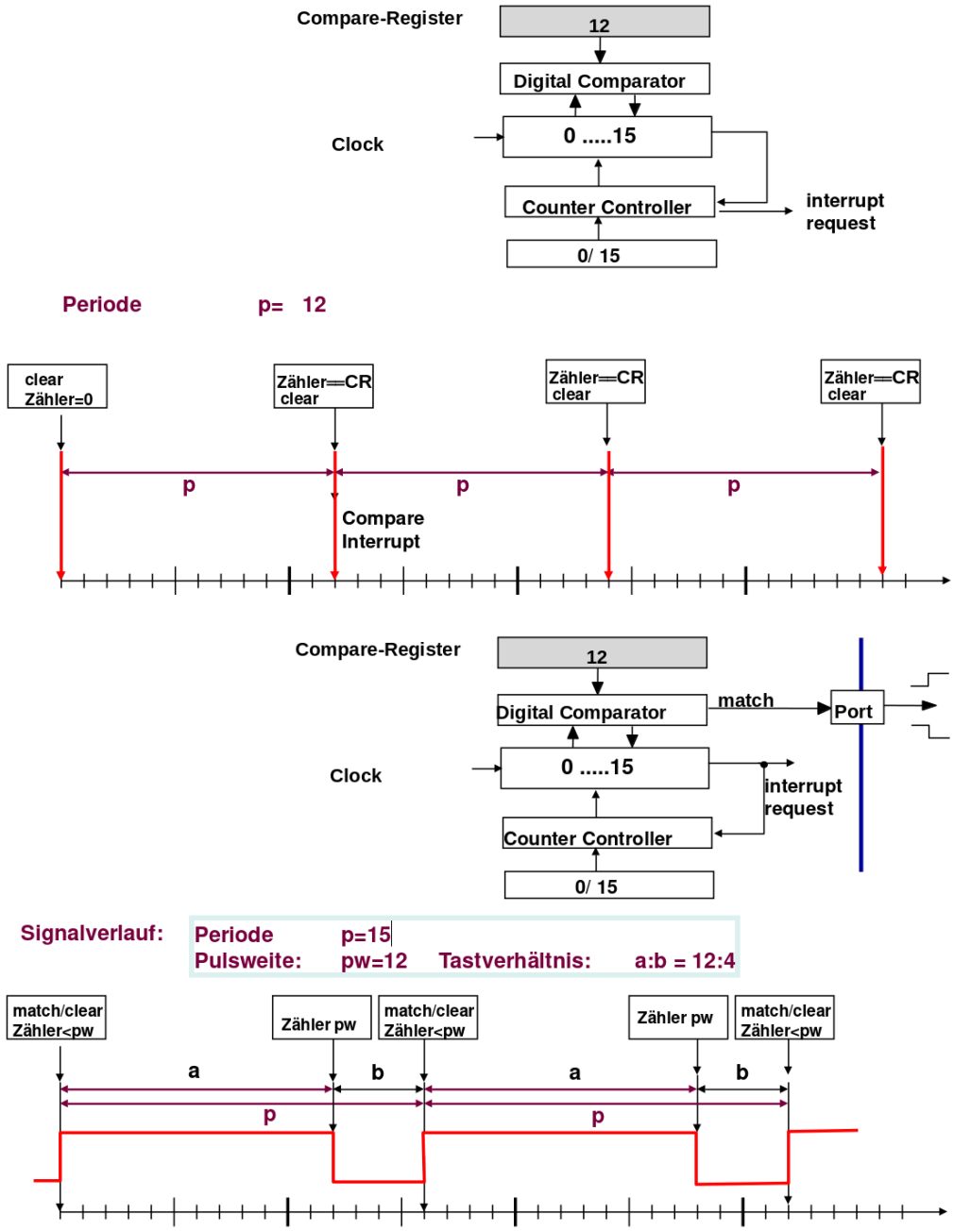


- Compare-Modus: Der Zählerstand des zugeordneten Timers wird mit dem eines Registers verglichen. Bei Übereinstimmung kann ein Interrupt ausgelöst werden.



Merke: Diese Vorgänge laufen ausschließlich in der Peripherie-Hardware ab, beanspruchen also, abgesehen von eventuellen Interrupts, keine Rechenzeit.

Entsprechend ergibt sich für den Compare-Modus verschiedene Anwendungen und ein zeitliches Verhalten entsprechend den nachfolgenden Grafiken.



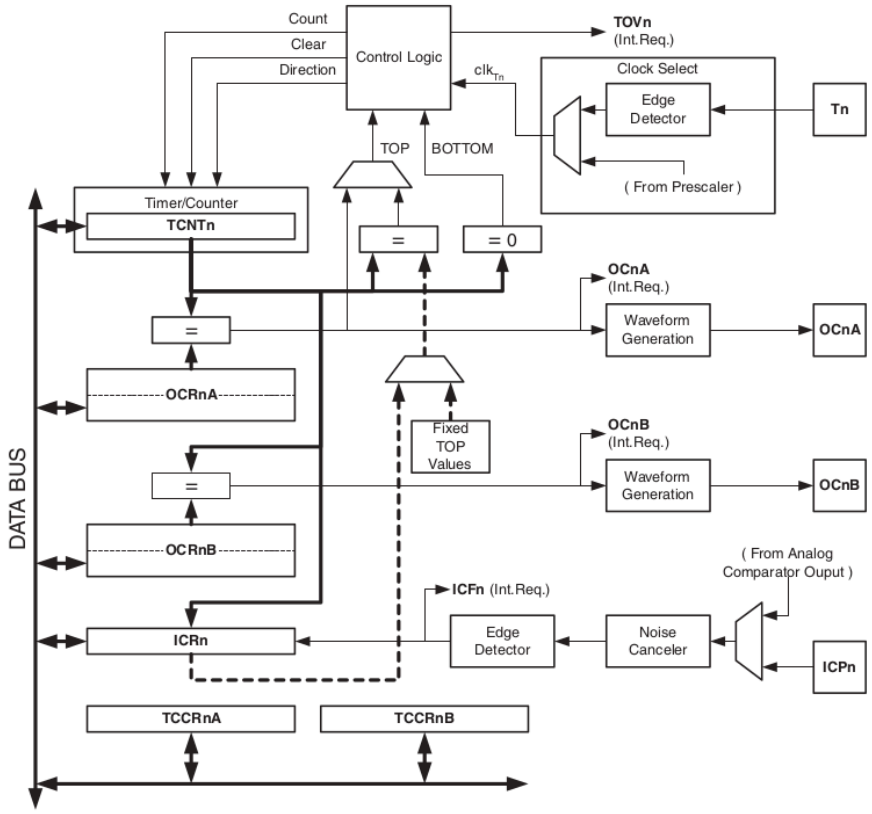
Umsetzung im AVR

Der AtMega328 bringt 4 unabhängige Timersysteme mit:

- *Two 8-bit Timer/Counters with Separate Prescaler and Compare Mode*
- *One 16-bit Timer/Counter with Separate Prescaler, Compare Mode, and Capture Mode*
- *Real Time Counter with Separate Oscillator*
- *Six PWM Channels*

Dabei soll die Aufmerksamkeit zunächst auf dem 16-bit Timer/Zähler liegen.

Figure 16-1. 16-bit Timer/Counter Block Diagram[1]



Timer Baustein im AVR Controller, Seite 250 [megaAVR]

Folgende Fragen müssen wir die Nutzung des Timers beantwortet werden:

- Wer dient als Trigger?
- Wie groß wird der Prescaler gewählt?
- Welche maximalen und die minimalen Schranken des Zähler werden definiert?
- Wird ein Ausgang geschaltet?

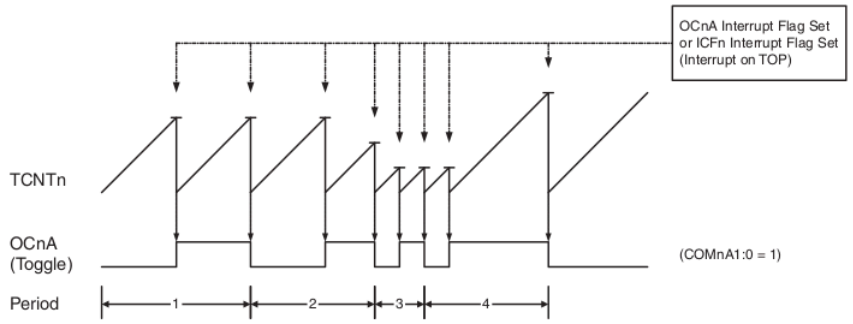
[megaAVR] Firma Microchip, megaAVR® Data Sheet, Link

Timer Modi

Timer-Modi bestimmen das Verhalten des Zählers und der angeschlossenen Ausgänge / Interrupts. Neben dem als Normal-Mode bezeichneten Mechanismus existieren weitere Konfigurationen, die unterschiedliche Anwendungsfelder bedienen.

Clear to Compare Mode (CTC)

Figure 16-6. CTC Mode, Timing Diagram



CTC mode, Seite 141 [megaAVR]

Die Periode über eine OCnA Ausgang ergibt sich entsprechend zu

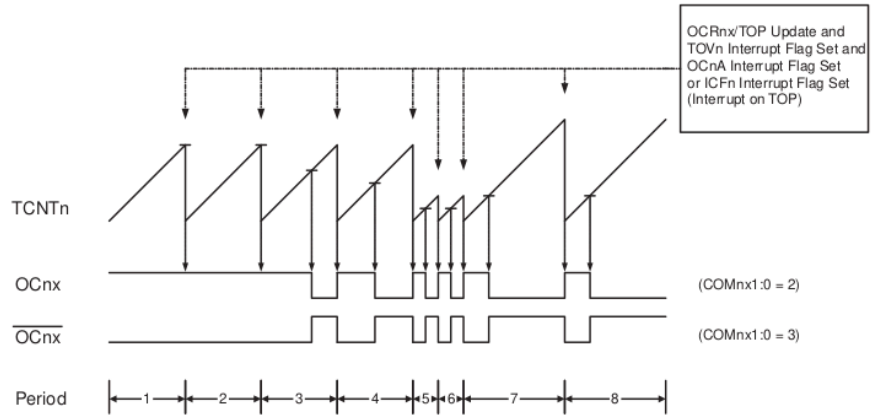
$$f_{OCnA} = \frac{f_{clk_i/o}}{2 \cdot N \cdot (1 + OCRnA)}$$

fOCnA = 2 · N · (1 + OCRnA)fclki/o

Der Counter läuft zwei mal durch die Werte bis zum Vergleichsregister OCRnA. Die Frequenz kann durch das Setzen eine Prescalers korrigiert werden.

Fast PWM

Figure 16-7. Fast PWM Mode, Timing Diagram



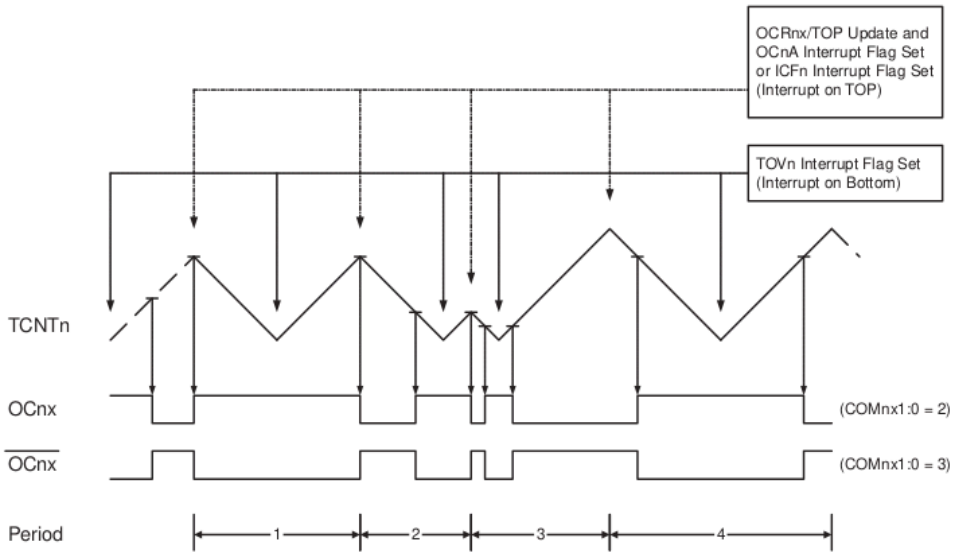
Fast PWM generation, Seite 141 [megaAVR]

Die Periode des Signals an OCRnA wechselt während eines Hochzählens des Counters. Damit kann eine größere Frequenz bei gleicher Auflösung des Timers verglichen mit CTC erreicht werden.

$$f_{OCnA} = \frac{f_{clk_i/o}}{N \cdot (1 + TOP)}$$

fOCnA = N · (1 + TOP)fclki/o

Phase Correct PWM



Phase correct PWM generation, Seite 141 [megaAVR]

$$f_{OCnA} = \frac{f_{clk_i/o}}{2 \cdot N \cdot (TOP)}$$

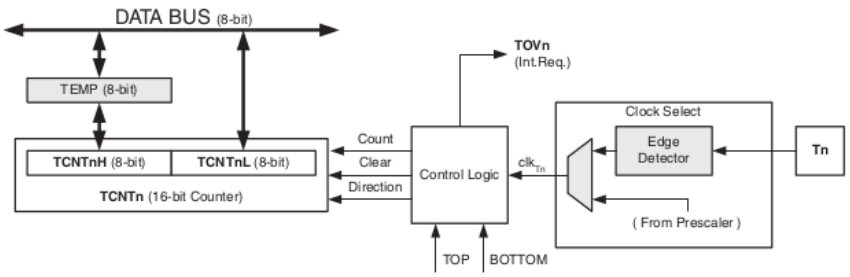
fOCnA = 2 · N · (TOP)fclki/o

[megaAVR] Firma Microchip, megaAVR® Data Sheet, Link

Timer-Funktionalität (Normal-Mode)

Für die Umsetzung eines einfachen Timers, der wie im nachfolgenden Beispiel jede Sekunde aktiv wird, genügt es einen entsprechenden Vergleichswert zu bestimmen, den der Zähler erreicht haben muss.

Figure 16-2. Counter Unit Block Diagram



Counter implementation, Seite 126 [megaAVR]

Simulation time: 00:18.789

13 **TCNT1:** 0x84,2 bytes, littleendian: 0011000000100101

avrlibc.cpp

```cpp
#ifndef F_CPU
#define F_CPU 16000000UL // 16 MHz clock speed
#endif

//16.000.000 Hz / 1024 = 15.625

int main(void)
{
  DDRB |= (1 << PB5);  // Ausgabe LED festlegen
  // Timer 1 Konfiguration
  //         keine Pins verbunden
  TCCR1A  = 0;
  TCCR1B  = 0;
  // Timerwert
  TCNT1   = 0;
  TCCR1B |= (1 << CS12) | (1 <<CS10);  // 1024 als Prescaler-Wert

  while (1) //infinite loop
  {
    if (TCNT1>15625){
      TCNT1 = 0;
      PINB = (1 << PB5); // LED ein und aus
    }
  }
}
```

```
starting simmulation...
compiling...
ready!
```

Was stört Sie an dieser Umsetzung?

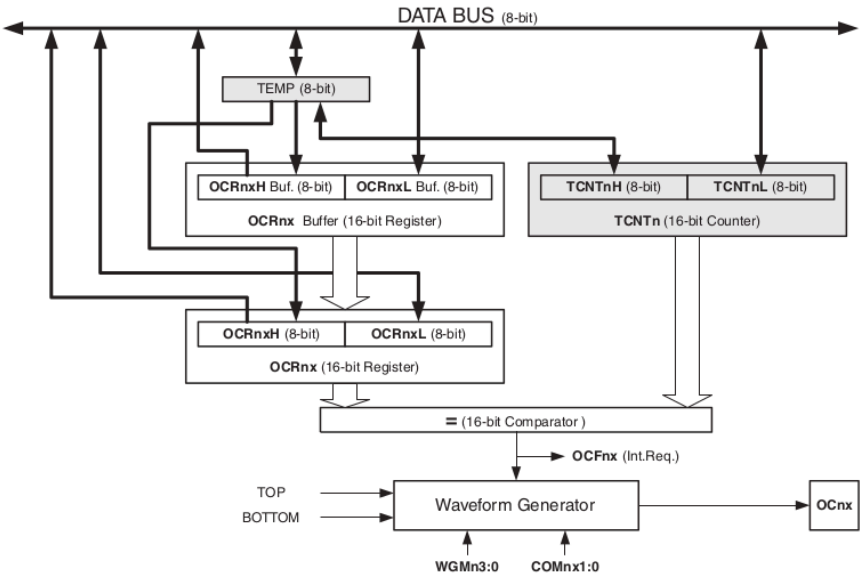
Wir lassen den Controller den Vergleichswert kontinuierlich auslesen. Damit haben wir noch nichts gewonnen, weil der Einsatz der Hardware unser eigentliches System nicht entlastet. Günstiger wäre es, wenn wir ausgehend von unseren Zählerzuständen gleich eine Schaltung des Ausganges vornehmen würden.

[megaAVR] Firma Microchip, megaAVR® Data Sheet, Link

Compare Mode

Wir verknüpfen unseren Timer im Comparemodus mit einem entsprechenden Ausgang und stellen damit sicher, dass wir die Ausgabe ohne entsprechende Ansteuerung im Hauptprogramm aktivieren.

Figure 16-4. Output Compare Unit, Block Diagram



Outputmode des Timerbausteins, Seite 126, [megaAVR]

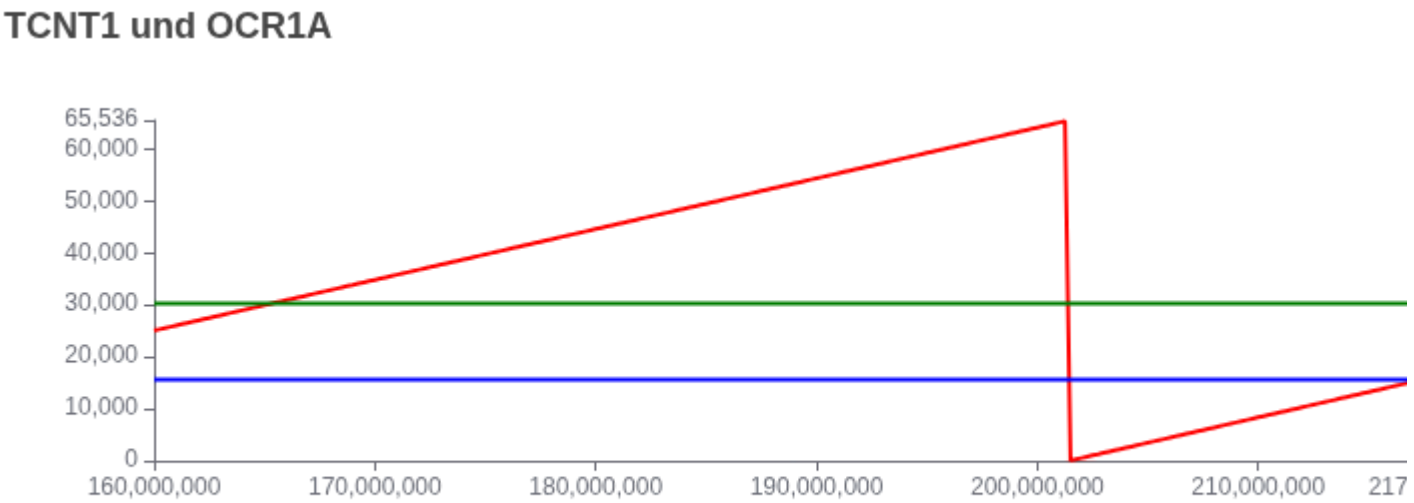
> **Frage:** Welchen physischen Pin des Controllers können wir mit unserem Timer 1 ansteuern?

Setzen wir also die Möglichkeiten des Timers vollständig ein, um das Blinken im Hintergrund ablaufen zu lassen. Ab Zeile 13 läuft dieser Prozess komplett auf der Hardware und unser Hauptprogramm könnte eigenständige Aufgaben wahrnehmen.

Normal Mode Konfiguration



Simulation time: 00:13.624



avrlibc.cpp

```cpp
#ifndef F_CPU
#define F_CPU 16000000UL // 16 MHz clock speed
#endif

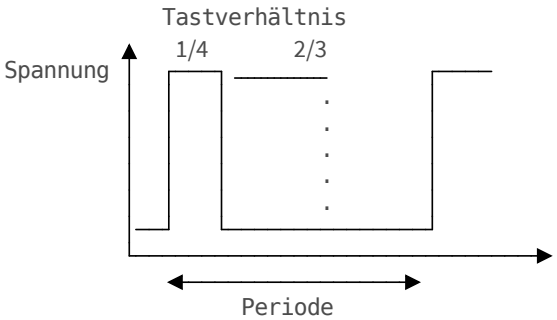
int main(void)
{
  DDRB |= (1 << PB1);  // Ausgabe LED festlegen
  TCCR1A = 0;
  TCCR1B = 0;
  TCCR1B |= (1 << CS12) | (1 <<CS10);  // 1024 als Prescale-Wert
  TCCR1A |= (1 << COM1A0);
  OCR1A = 15625;

  while (1) _delay_ms(500);
}
```

```
starting simmulation...
compiling...
ready!
```

Was passiert, wenn die Aktivierung und Deaktivierung mit einer höheren Frequenz vorgenommen wird? Die effektiv wirkende Spannung wird durch den Mittelwert repräsentiert. Damit ist eine Quasi-Digital-Analoge Ausgabe ohne eine entsprechende Hardware möglich.

> **Merke:** Reale Analog-Digital-Wandler würden ein Ergebnis zwischen 0 und 2^n2n projiziert auf eine Referenzspannung ausgeben. PWM generiert diesen Effekt durch ein variierendes Verhältnis zwischen an und aus Phasen.



Im folgenden wird der **Fast PWW Mode** genutzt, um auf diesem Wege die LED an PIN 9 zu periodisch zu dimmen. Dazu wird der Vergleichswert, der in OCR1A enthalten ist kontinuierlich verändert.

avrlibc.cpp

```cpp
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void){
    DDRB |= (1<<PB1);           // PB1 = Pin 9

    TCCR1A = (1<<COM1A1) | (1<<WGM10);
    TCCR1B = (1<<WGM12) | (1<<CS11);

    int timer = 0;

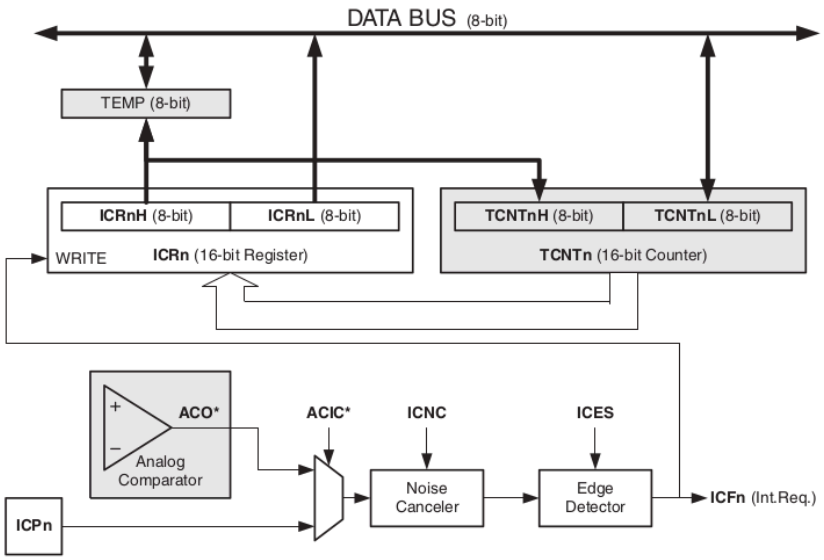
    while (1) {
        while (timer < 255) {
            OCR1A = timer++;
            _delay_ms(10);
        }
        while (timer > 0) {
            OCR1A = timer--;
            _delay_ms(10);
        }
    }
}
```

Timer1, Fast PWM (8-bit), Mode 5, non-inverting, Prescaler /8

[megaAVR] Firma Microchip, megaAVR® Data Sheet, Link

Capture Mode

Figure 16-3. Input Capture Unit Block Diagram



Capture Mode, Seite, 126, [megaAVR]

avrlibc.cpp

```
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
  Serial.begin(9600);
  DDRB = 0;
  PORTB = 0xFF;

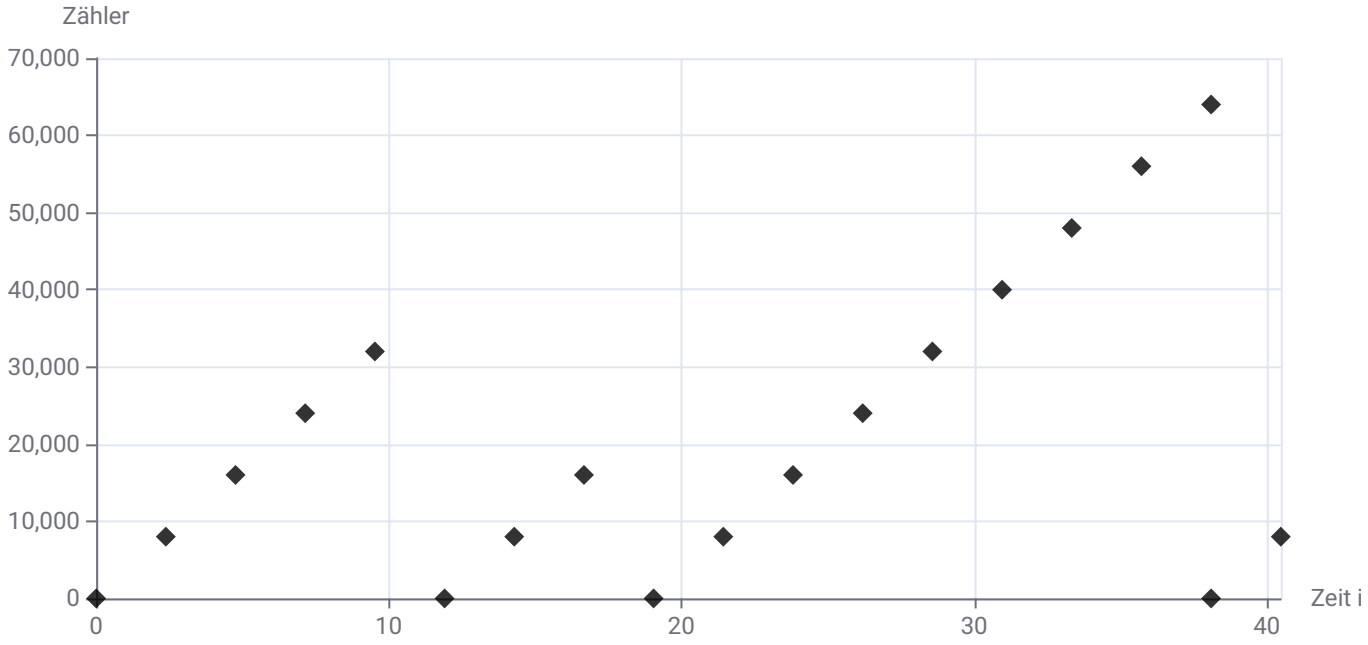
  TCCR1A = 0; // Normal Mode
  TCCR1B = 0;
  //         1024 als Prescale-Wert     Rising edge
  TCCR1B  = (1 << CS12) | (1 <<CS10) | (1 << ICES1);
  TIFR1 = (1<<ICF1);  // Löschen des Zählerwertes

  while (1)
  {
      Serial.println("Waiting for Button push");
      Serial.flush();
      TCNT1 = 0;  // Löschen des Zählerwertes
      while ((TIFR1 & (1<<ICF1)) == 0);
      TIFR1 = (1<<ICF1);
      Serial.println(ICR1);
      Serial.flush();
      _delay_ms(500);
  }
  return  0;
}
```

Mit dem Schreiben des Flags `ICF1` kann der Eintrag gelöscht werden.

_This flag is set when a capture event occurs on the ICP1 pin. When the Input Capture Register (ICR1) is set by the WGM13:0 to be used as the TOP value, the ICF1 Flag is set when the counter reaches the TOP value. ICF1 is automatically cleared when the Input Capture Interrupt Vector is executed. Alternatively, ICF1 can be cleared by writing a logic one to its bit location.

Entwicklung des Timerinhalts



Frage: Sie wollen die Ausgabe in Ticks in eine Darstellung in ms überführen. Welche Kalkulation ist dafür notwendig?

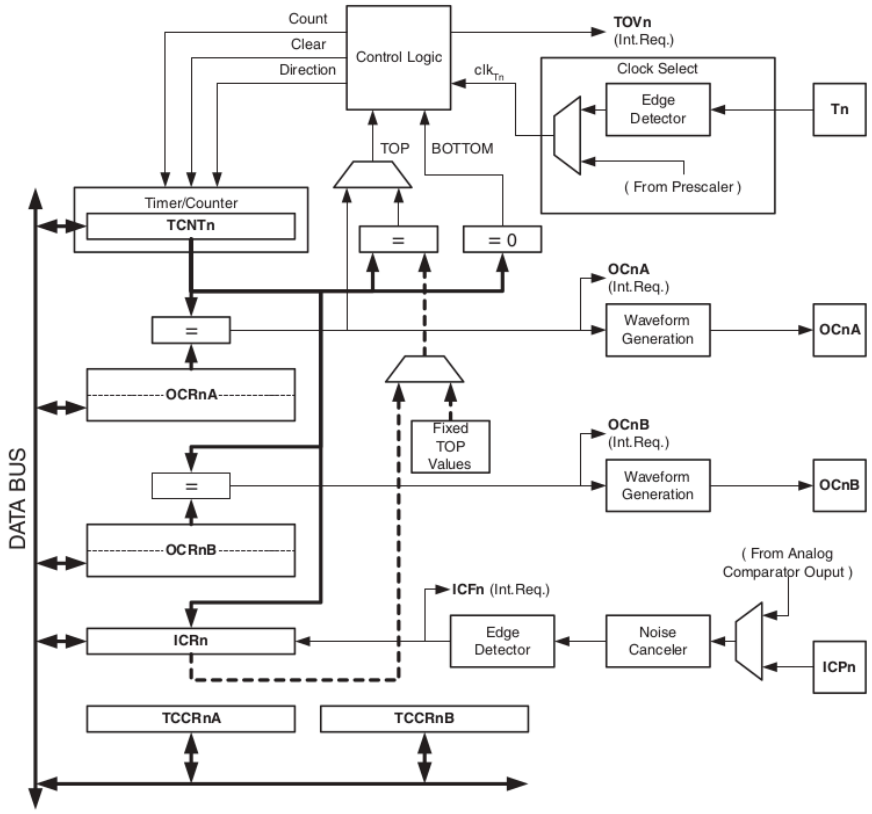
Problem: Wie groß ist das maximal darstellbare Zahlenintervall?

[megaAVR] Firma Microchip, megaAVR® Data Sheet, Link

Anwendungen

Eingangszähler

Figure 16-1. 16-bit Timer/Counter Block Diagram[(1)]



Übersicht Timermodul Seite 250, [megaAVR]

Wir wollen einen Eingangszähler entwerfen, der die Ereignisse als Zählerimpulse betrachtet und zusätzlich mit einem Schwellwert vergleicht.

avrlibc.cpp

```
#include <avr/io.h>
#include <util/delay.h>

ISR (TIMER1_COMPA_vect)
{
  Serial.print("5 Personen gezählt");
  TCNT1 = 0;
}

int main(void)
{
  Serial.begin(9600);
  TCCR1A = (1 << WGM12);
  // CTC Modus (Fall 4)
  TCCR1B |= (1 << CS12) | (1 << CS11) | (1 << CS10);
  OCR1A = 5;
  TIMSK1 |= (1<<OCIE1A);
  sei();

  while (1)
  {
    Serial.print(TCNT1);
    Serial.print(" ");
    _delay_ms(500);
  }
  return  0;
}
```

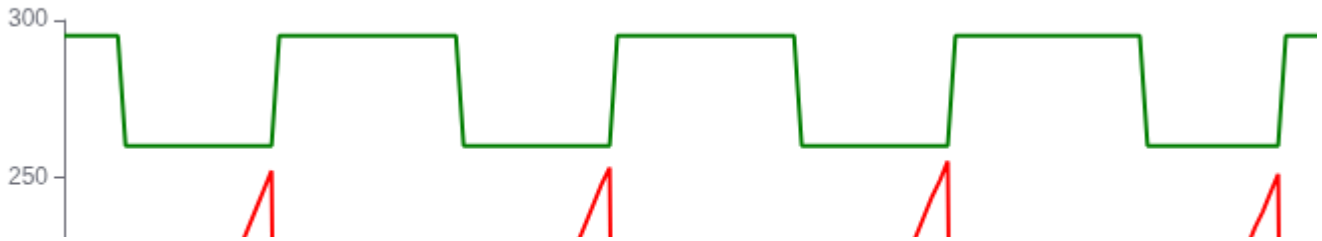
Glimmeffekt einer LED

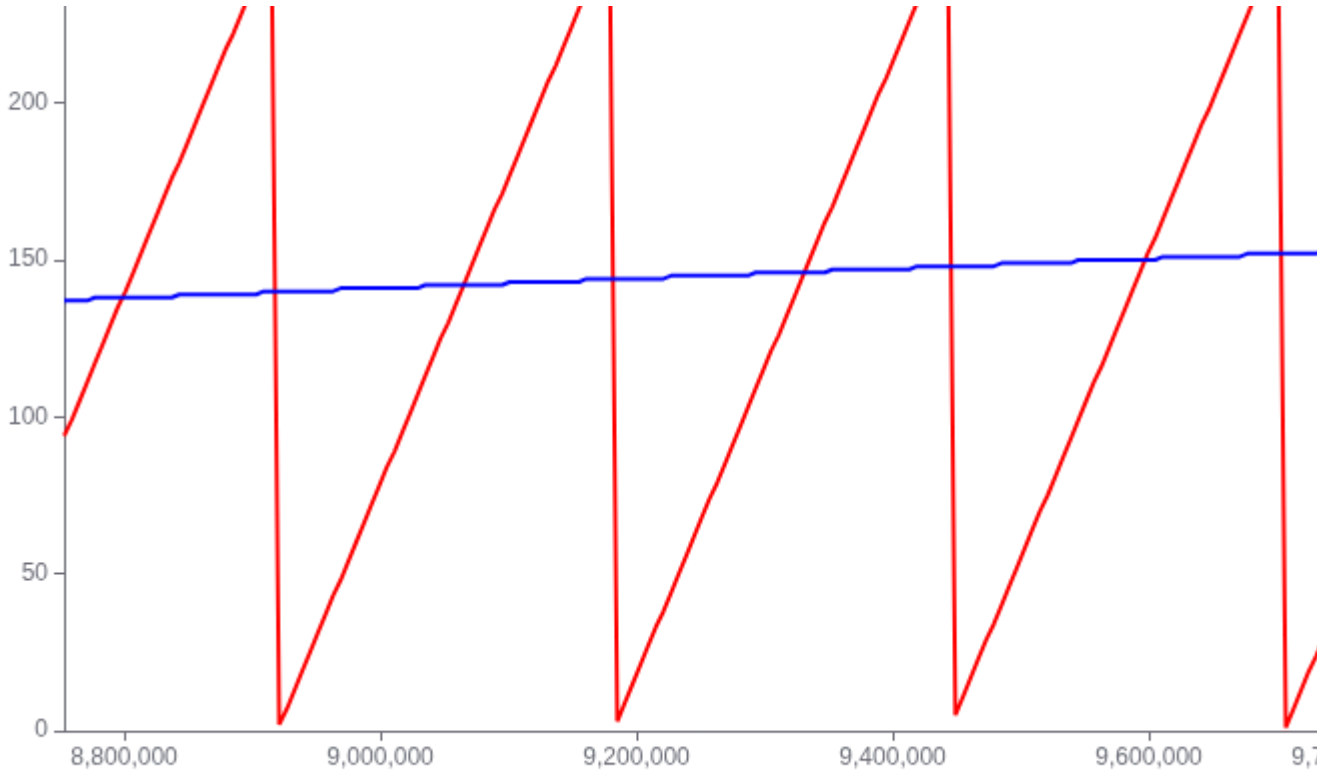
Simulation time: 00:00.609



PORTB: 0x25: 00000000 **DDRB:** 0x24: 00000010 **PINB:** 0x23: 00000010
TCCR1: 0x80,2 bytes, littleendian: 0000110110000001
TCNT1: 0x84,2 bytes, littleendian: 0000000000110000
OCR1A: 0x88,2 bytes, littleendian: 0000000010011001

TCNT1 und OCR1A




200
150
100
50
0
8,800,000
9,000,000
9,200,000
9,400,000
9,600,000

Dimmer.cpp

```cpp
#ifndef F_CPU
#define F_CPU 16000000UL // 16 MHz clock speed
#endif

int main(void){
  DDRB |=  (1 << PORTB1); //Define OCR1A as Output
  TCCR1A |= (1 << COM1A1) | (1 << WGM10);  //Set Timer Register
  TCCR1B |= (1 << CS12) | (1 << CS10) | (1 << WGM12);
  OCR1A = 0;
  int timerCompareValue = 0;
  while(1) {
   while(timerCompareValue < 255){ //Fade from low to high
    timerCompareValue++;
    OCR1A = timerCompareValue;
    _delay_ms(4);

  }
   while(timerCompareValue > 0){ //Fade from high to low
    timerCompareValue--;
    OCR1A = timerCompareValue;
    _delay_ms(4);
    }
   }
}
```

```
starting simmulation...
compiling...
ready!
```

[megaAVR] Firma Microchip, megaAVR® Data Sheet, Link

Was Sie heute mitnehmen

Diese fünf Punkte fassen die Vorlesung zusammen: Interrupts lassen den Controller auf Ereignisse reagieren, Timer erledigen Zeitaufgaben in der Hardware. Die Fragen darunter gehören zu den Lernzielen vom Anfang; rechnen Sie die Frequenz eines Timer-Signals mindestens einmal selbst nach.

- **Polling oder Interrupt:** Beim Polling fragt das Programm ständig ab und belastet den Prozessor; ein **Interrupt** unterbricht das Hauptprogramm nur, wenn das Ereignis eintritt. Der laufende Befehl wird beendet, der Zustand auf dem Stack gesichert, über die **Vektortabelle** in die **ISR** gesprungen und danach zurückgekehrt.
- **Zweistufige Freigabe beim ATmega328P:** Eine ISR läuft nur, wenn das I-Bit im SREG gesetzt ist, das Maskenbit der Quelle gesetzt ist und das Ereignis eintritt. Pro Quelle merkt sich der Controller nur **ein** ausstehendes Ereignis – ISRs müssen deshalb kurz bleiben.
- **Gemeinsame Daten:** Variablen, die Hauptprogramm und ISR teilen, sind `volatile`; Lese-Ändere-Schreib-Zugriffe darauf sichern Sie mit `ATOMIC_BLOCK` oder `cli()`/`sei()`.
- **Ein Timer** ist ein Zähler mit Prescaler. Die **Compare-Einheit** vergleicht den Zählerstand mit einem Register und schaltet einen Ausgang oder löst einen Interrupt aus, die **Capture-Einheit** speichert den Zählerstand bei einem externen Ereignis – beides ohne Rechenzeit der CPU.
- **PWM:** Im CTC-, Fast-PWM- oder Phase-Correct-Modus erzeugt der Timer Pulsfolgen am Ausgangspin; das **Tastverhältnis** bestimmt die mittlere Spannung, die Frequenz folgt aus Takt, Prescaler und Vergleichs- bzw. TOP-Wert.

Prüfen Sie sich:

- Nennen Sie je einen Vor- und Nachteil von Polling und Interrupts.
- Welche Schritte werden bei der Abarbeitung eines Interrupts durchlaufen?
- Welche drei Bedingungen müssen erfüllt sein, damit der ATmega328P eine ISR ausführt?
- Nennen Sie vier Interruptquellen des ATmega328P.
- Wie oft kann sich der ATmega328P dasselbe Interruptereignis vormerken, während eine ISR läuft, und was folgt daraus für die Dauer einer ISR?
- Warum ist eine Anweisung wie `port |= 0x03` gefährlich, wenn auch eine ISR auf `port` zugreift, und wie sichern Sie sie ab? Wozu dient `volatile`?
- Für welche Aufgaben werden Timer in einem Mikrocontroller eingesetzt?
- Wie ist ein Timer grundlegend aufgebaut, und was leisten Capture- und Compare-Einheit?
- Was ist PWM, und mit welchen Timer-Modi erzeugt der ATmega328P sie?
- Timer 1 läuft im CTC-Modus mit 16 MHz, Prescaler 1024 und OCR1A = 15624 und schaltet den Ausgang bei jedem Vergleich um. Welche Frequenz hat das Signal an OC1A?

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester, und Sie bestimmen mit, was hineinkommt. Ihre Antworten sammeln wir im Hörsaal gemeinsam über das LiaScript-Klassenzimmer; nach der Vorlesung werden die Begriffe dieser Woche in der Karte freigeschaltet.

Welche Begriffe aus der heutigen Vorlesung gehören für Sie in die Kurslandkarte?

Texteingabe ...

Wo begegnet Ihnen das, was heute dran war – in Ihrem Studiengang oder in Geräten Ihres Alltags?

Texteingabe ...